

Baseball KATA

CONTENTS

목차

Chapter1

Baseball 소개

Chapter2

Baseball 프로젝트 준비

Chapter3

Baseball TDD 개발

Chapter1

Baseball 소개

Baseball Game 소개

- 세 개의 숫자를 맞추는 게임

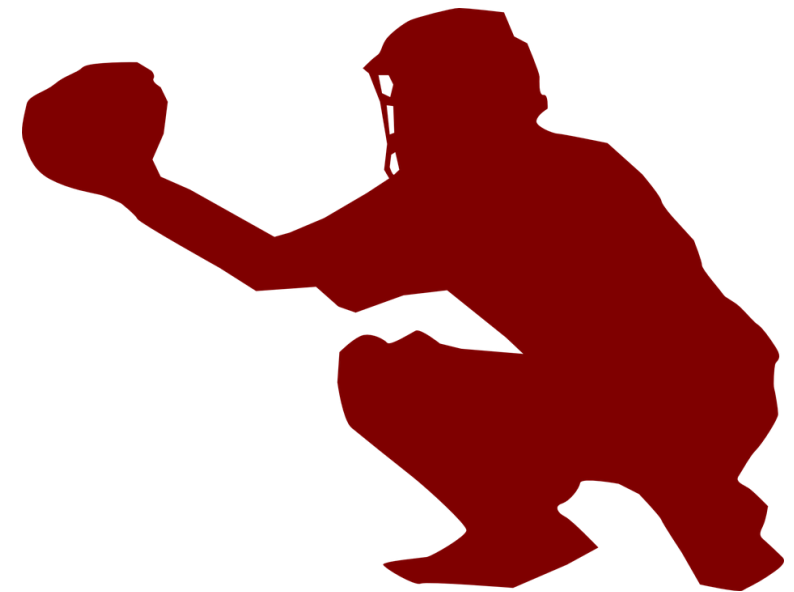
세 자리 숫자를 시도할 때 마다,
정답인지 아닌지 알려주며
정답을 유추할 수 있도록 힌트를 준다.

- 힌트를 주는 방식

Strike / Ball의 개수를 알려준다.

Ball : 정답과 일치하는 숫자의 개수

Strike : 정답과 일치하는 숫자이면서, 위치까지 정한 숫자의 개수



Baseball Game 예시

- 다음은 정답이 "123" 일 때 예시이다.

guess("456")

-> solved = **false**, strikes = 0, balls = 0

guess("129")

-> solved = **false**, strikes = 2, balls = 0

guess("240")

-> solved = **false**, strikes = 0, balls = 1

guess("321")

-> solved = **false**, strikes = 1, balls = 2

guess("123")

-> solved = **true**, strikes = 3, balls = 0



Chapter2

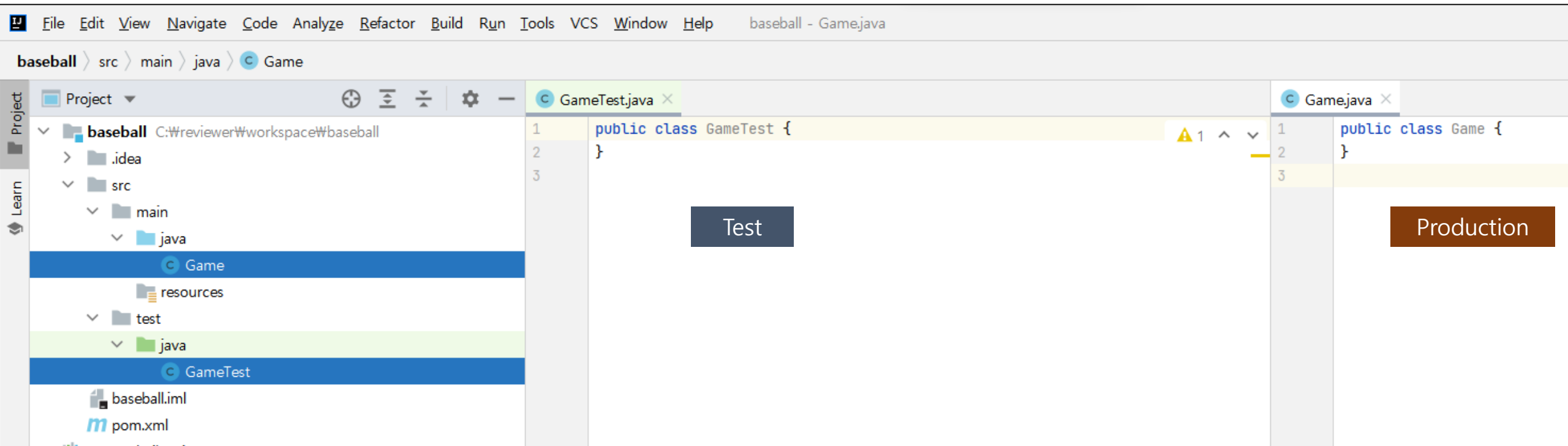
Baseball 프로젝트 준비

프로젝트 생성 후 파일 배치

Game.java 파일 생성

왼쪽 : GameTest.java

오른쪽 : Game.java



Remote Repository 생성

• Github Repository 생성하기

- Repo. name : Baseball-번호
- Description : 이름
- README.md 파일 **생성 안함**

생성 후 설정

- **Setting > Collaborator** : 팀원 추가하기

Remote Repository 에서는 Collaborator만 코드리뷰어 활동이 가능합니다.

Remote Repository에
팀원들 모두 **Collaborator**로 등록해주세요

github.com/new

Search or jump to...

Pull requests Issues Codespaces Marketplace Explore

Create a new repository

A repository contains all project files, including the revision history. Already have a project? [Import a repository.](#)

Owner * Repository name *
Reviewer-XX / **Baseball-12345**

Great repository names are short and memorable. Need inspiration? How about [ficti](#)

Description (optional)
이름

☐ Public Any logged in user can see this repository. You choose who can commit.

☒ Internal Samsung SDS enterprise members can see this repository. You choose who can commit.

☐ Private You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ Add a README file This is where you can write a brief description for your project. [Learn more.](#)

Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)

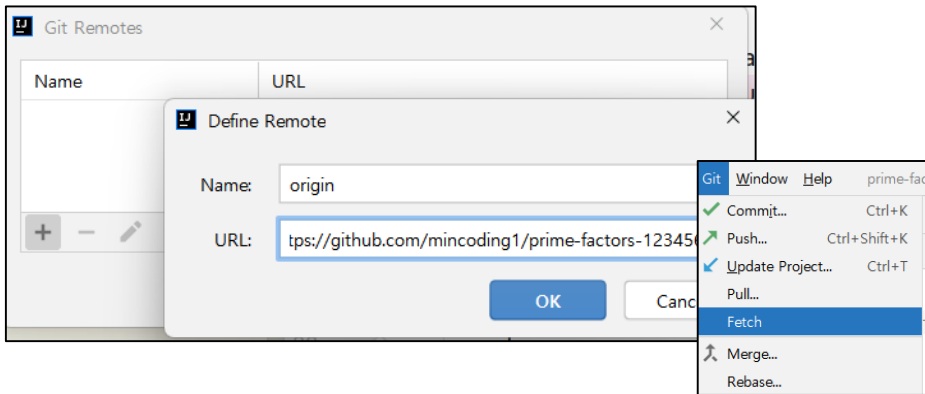
.gitignore template: **None**

Choose a license

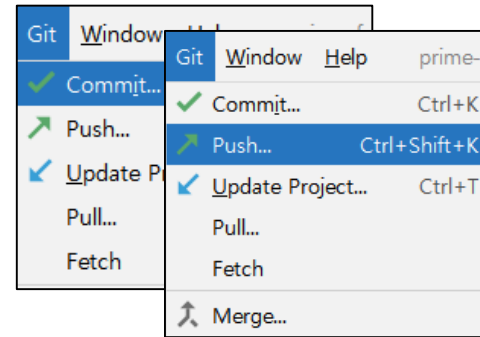
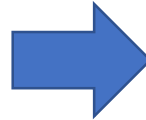
공개여부는 자유롭게 선택

현재까지 내용 Push를 위한 준비

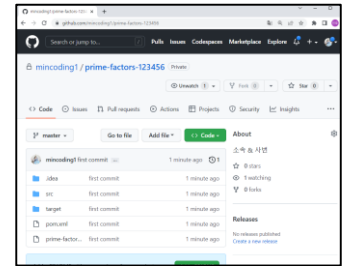
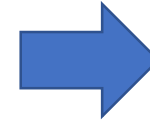
Git GUI 도구를 이용하여 Push를 해보자.



Git 주소 Remote 연결 후 Fetch



Commit 후 Push 하기
Commit 메시지 : "first commit"



Github 결과 확인

Github Repository 생성

Push 후 설정

- 팀원 추가

Settings > Collaborators and teams : 강사 / 팀원 추가하기

- **Branch protection rule**

Settings > Branches > Branch protection rules > Add rule 선택

master(main) 브랜치에 대해,

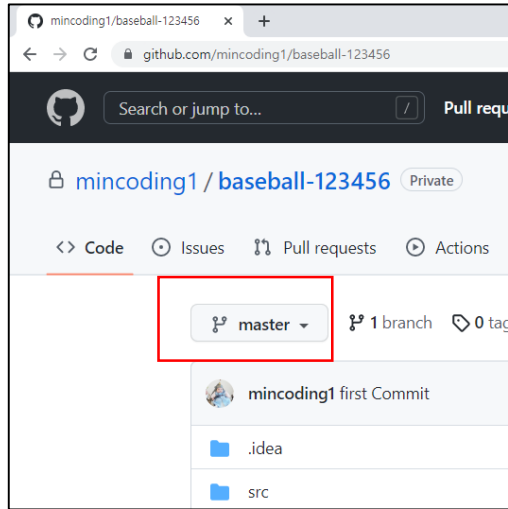
Require a pull request before merging 체크, Require approvals 체크

Required number of approvals before merging: 1 선택

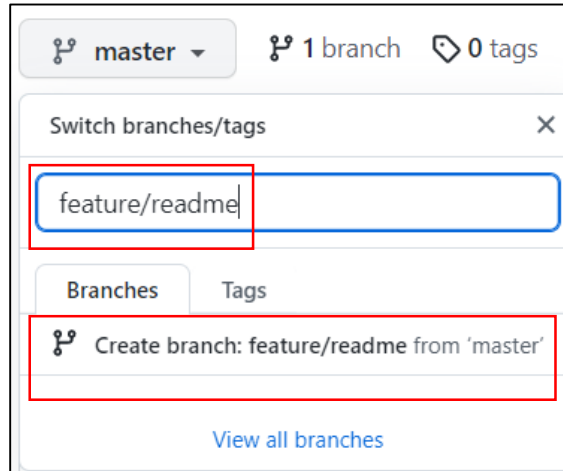
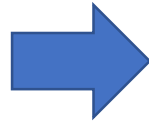
하단 초록색 save 버튼 클릭해서 설정 저장

Branch 하나 추가하기

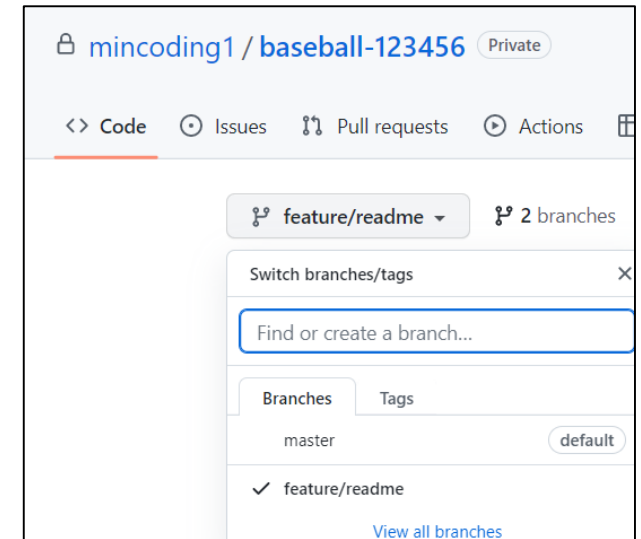
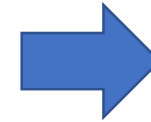
PR 테스트를 위해
브랜치 생성 : feature/readme



Branch 목록 클릭



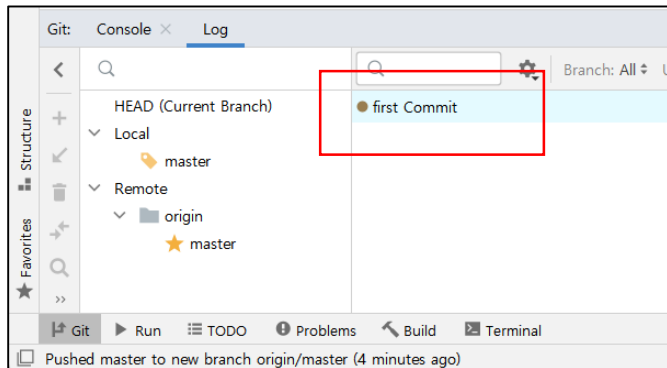
feature/readme 입력 후
Create 버튼 클릭



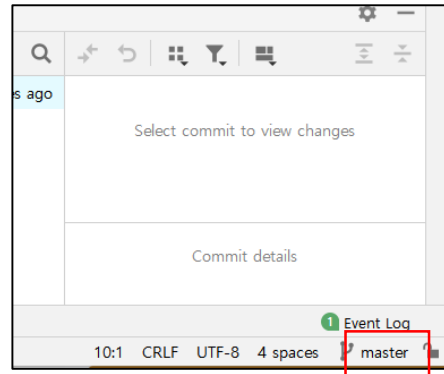
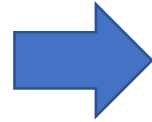
Branch 목록을 보면
2개가 생성됨을 알 수 있음

IntelliJ에서 현재 상태 확인하기

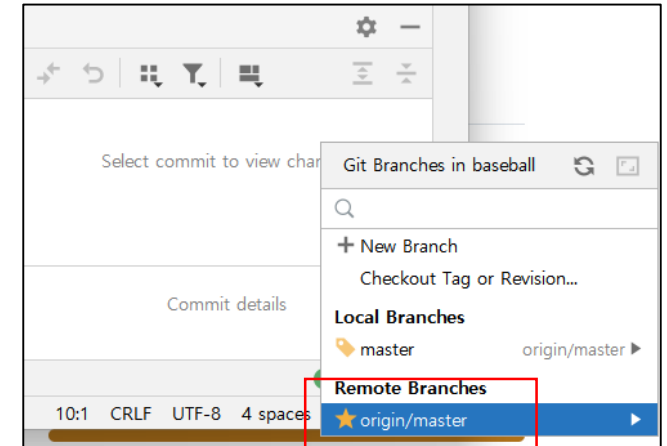
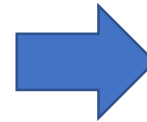
아직 Github과 Sync가 맞지 않는 상태임



Git > Log 를 보면
Commit 은 현재 1개 있음



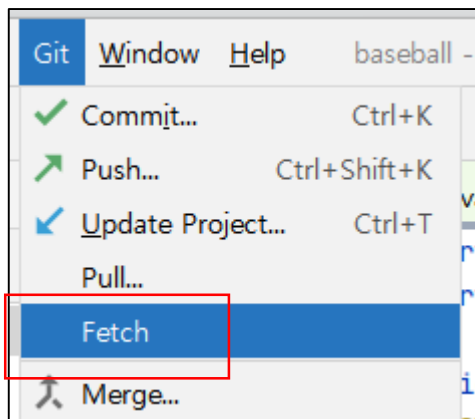
우측 최 하단
master 버튼 클릭



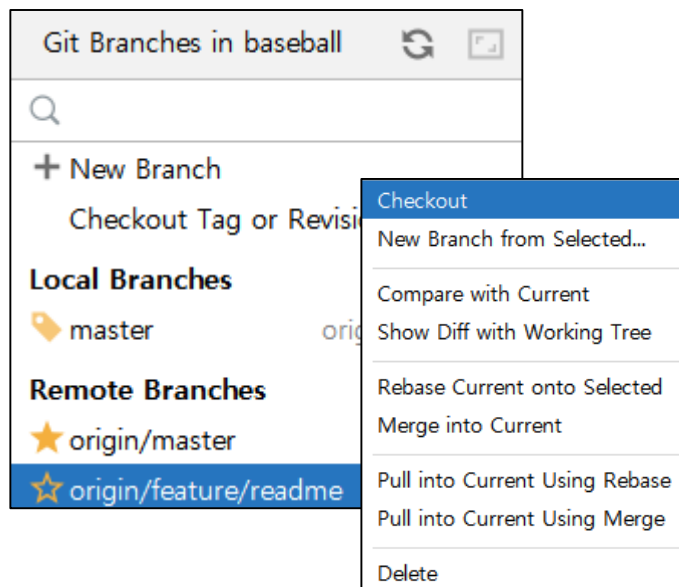
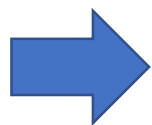
현재 Branch 는
master밖에 없음

Checkout 하기

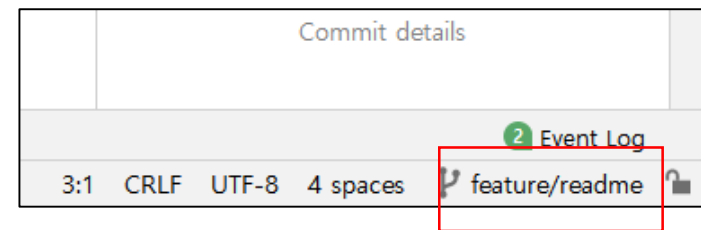
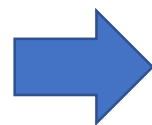
Fetch 하고 15초 기다리면,
생성했던 Branch 목록이 나온다.



Fetch 선택
10초 기다리자.



아까 만들었던
Branch 로 Checkout 하기



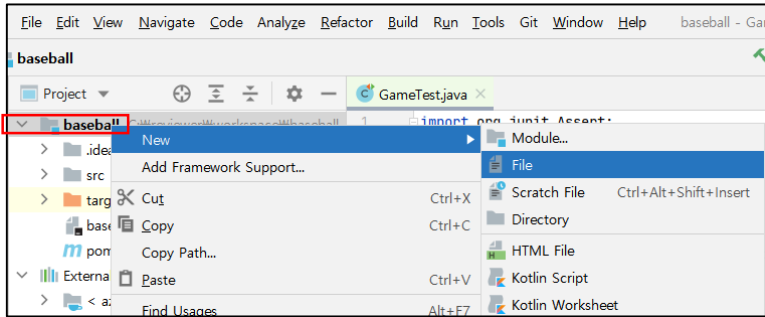
Branch 전환 완료

README.md 파일 생성

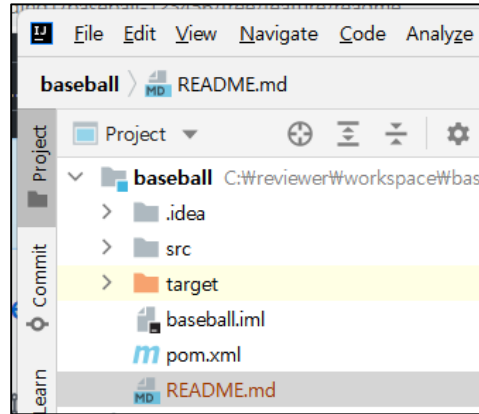
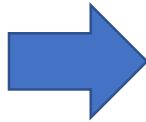
파일 생성하기

현재 master branch 에서 README.md 파일을 생성한 것이 아니다.

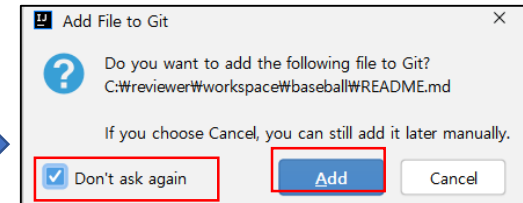
현재 feature/readme branch 에서 README.md 파일을 생성한 것이다.



프로젝트 최 상단 항목에서 마우스
우클릭 > New > File



README.md 파일 생성

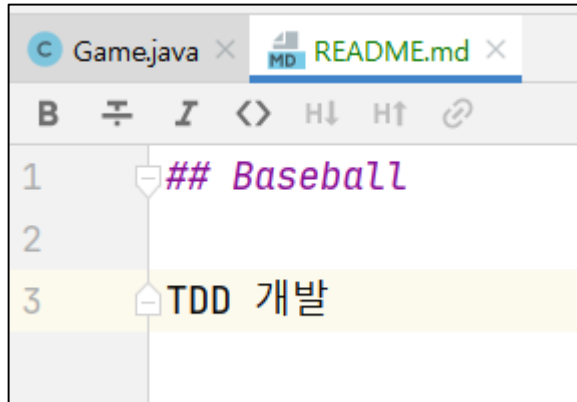


만약 Git Add를 직접 해줄까? 메
세지가 뜬다면

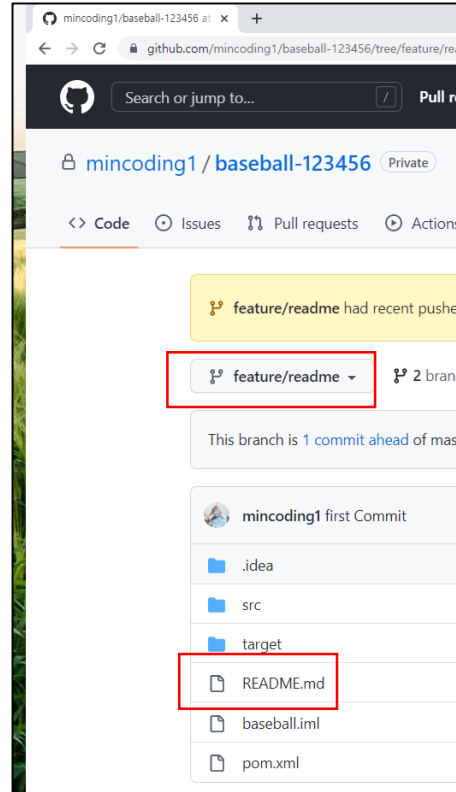
- 다신 묻지 않음
- Add 선택

Branch 확인

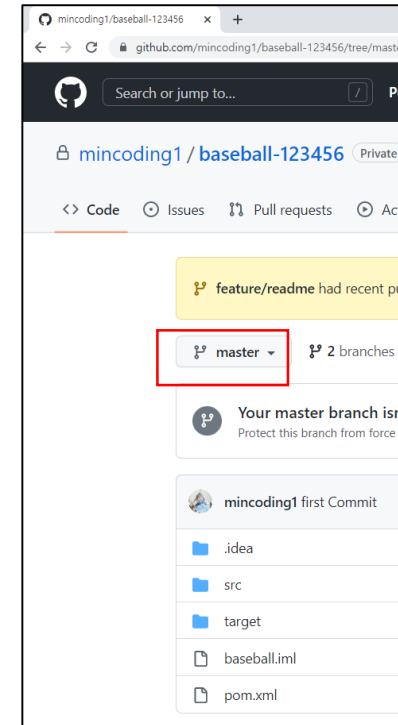
github에서 브랜치를 확인하기 전에 intellij에서 commit 생성 후 push를 해야 한다.



commit 후, push 한다.



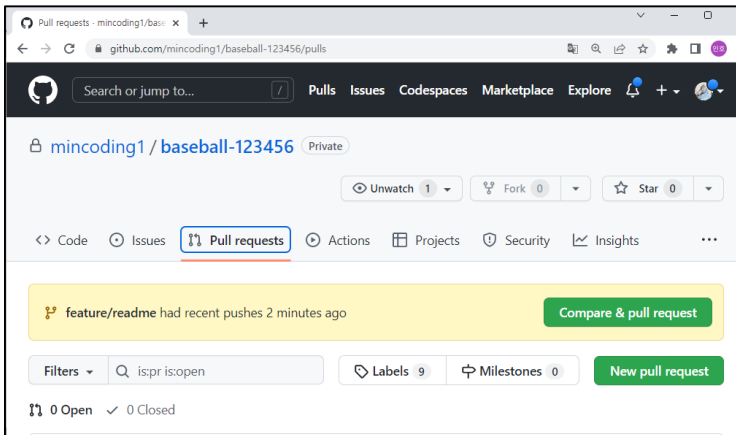
feature/readme 브랜치
README.md 파일 존재



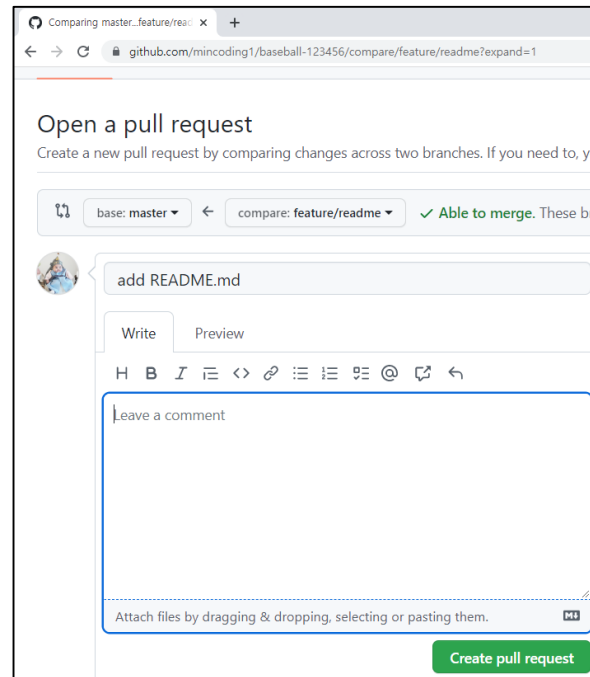
master 브랜치
README.md 파일 없음

PR Create

master Branch 와 병합을 요청하는 Pull Request를 Create 한다.
PR 요청을 위해 반영하고자 하는 내용에 대해 의미있는 메시지를 간단하게 입력한다.



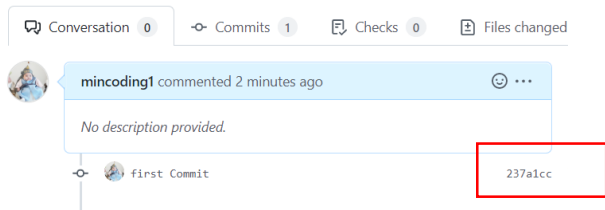
Pull Request 에서
PR 작성하기



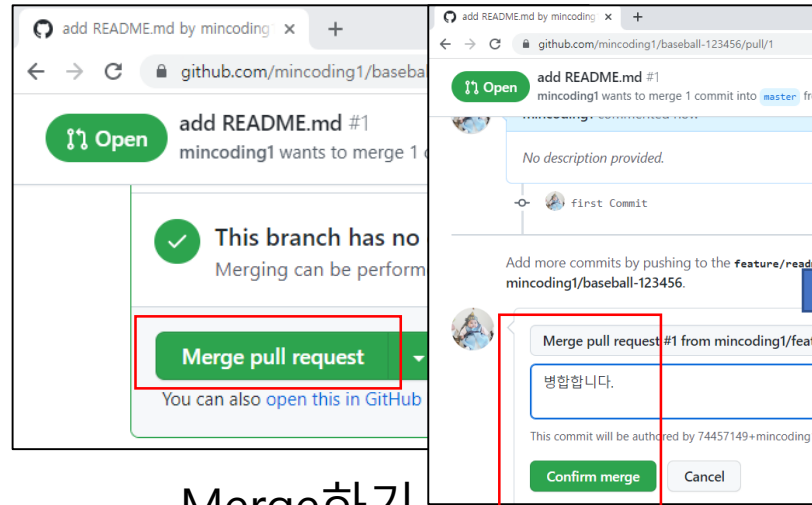
PR Create

Merge PR

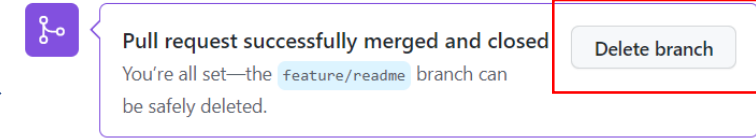
요청이 들어온 PR에 대해 어떤 내용인지 확인한다. (마치 다른 사람이 PR 요청을 한것 처럼)
Merge를 한다.



PR Commit 내용 확인



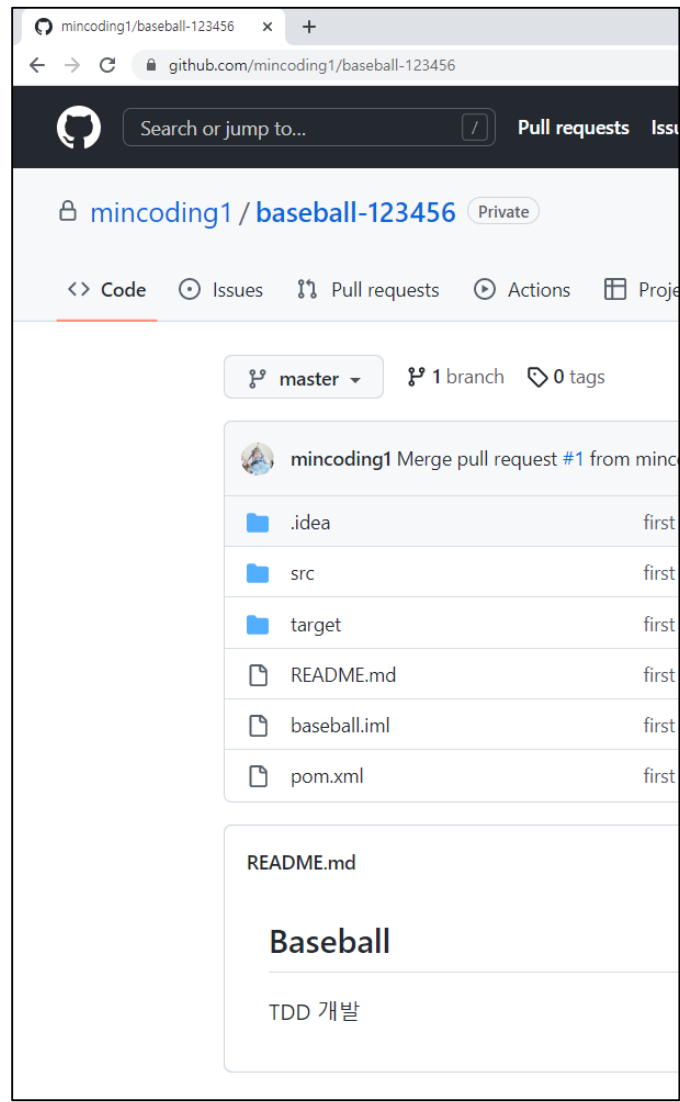
Merge하기



Merge 완료되었고
더이상 Branch가 필요없어
Delete하면 된다.

PR 후 Merge 확인

Merger가 성공적으로 잘 되어,
master branch에
README.md 파일이 존재하는지 확인한다.



실습 과정 구성

- 본 실습 과정은 총 8개의 TDD Cycle Step 이 있다.
- 각 Step 은 다음과 같이 나누어져 있다.

1. Red
2. Green
3. Refactor

- **Commit 타이밍**

1. Red 단계에서는 Commit 하지 않는다.
2. Green 모두 수행 후, Unit Test Pass 시 **Commit** 을 한다.
3. Refactor 모두 수행 후, Unit Test 를 수행하며, Pass 시 **Commit**을 한다.

- **Commit 메시지 헤더 Format**

- Red 단계 : Commit 하지 않는다. (동작되는 코드만 Commit 해야 하기 때문)
- Green 단계 : **[feature] page 번호**
- Refactor 단계 : **[refactoring] page 번호**

- **Push 하여 최종 제출하기**

8 단계 까지 완료 이후, Push 1회만 진행

Chapter3

Baseball TDD 개발

Baseball Game 실습(01/06)

TDD Cycle – Red 부터 시작

Red – 입력값이 없을 경우 테스트 케이스 작성

RED

```
class GameTest {  
    @Test  
    void createGame() {  
        Game game = new Game();  
        assertNotNull(game);  
    }  
  
    @Test  
    void throwExceptionWhenInputIsNull() {  
        assertThrows(IllegalArgumentException.class, () -> {  
            Game game = new Game();  
            game.guess(null);  
        });  
    }  
}
```

(GameTest.java)

IllegalArgumentException이 발생함을
기대하는 테스트 코드 작성

Green – 메서드 추가하기

TDD Cycle – Red 부터 시작

Red – 입력값이 없을 경우 테스트 케이스 작성

GREEN

Test

```
class GameTest {  
    @Test  
    void createGame() {  
        Game game = new Game();  
        assertNotNull(game);  
    }  
  
    @Test  
    void throwExceptionWhenInputIsNull() {  
        assertThrows(IllegalArgumentException.class, () -> {  
            Game game = new Game();  
            game.guess(null);  
        });  
    }  
}
```



Production

```
Game.java  
1 public class Game {  
2     public void guess(Object o) {  
3     }  
4 }  
5
```



자동생성 완료
(Game.java)

```
Run: GameTest  
Tests failed: 1, passed: 1 of 2 tests - 10 ms  
GameTest  
- throwExceptionWhenInputIsNull 10 ms  
+ createGame 0 ms  
java.lang.AssertionError: Expected exception  
<18 internal calls>  
Process finished with exit code -1
```

Exception이 발생 안되어
Fail 발생한다.

Fail을 눈으로 봐야한다.

Alt + Enter를 눌러
Create Method

Green – Exception 발생시키기

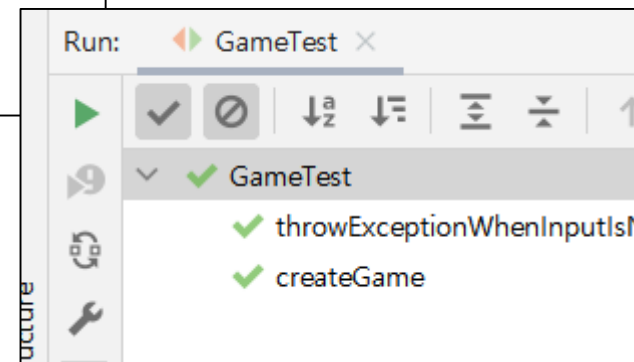
Green : 동작되는 코드로 만든다.

GREEN

```
public class Game {  
  
    public void guess(Object o) {  
        if (o == null) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```

Production

Game.java 코드 수정



Green을 눈으로 확인한다.

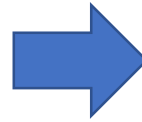
Refactoring – 네이밍 변경

변수명을 변경한다.

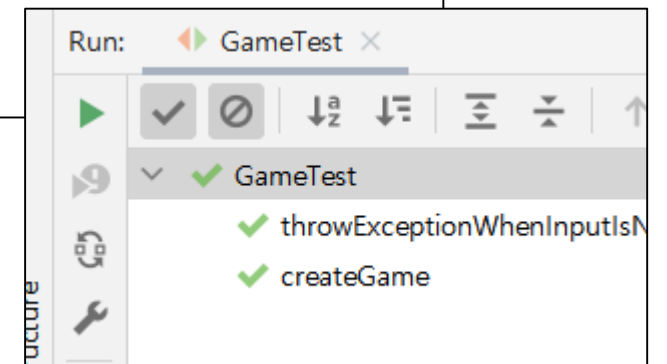
Refactor

```
public class Game {  
  
    public void guess(Object o) {  
        if (o == null) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```

Production



```
public class Game {  
  
    public void guess(Object guessNumber) {  
        if (guessNumber == null) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```



리팩토링 - 중복 코드 제거

테스트코드 중복 코드 제거를

다음 페이지를 보며, 왼쪽 소스코드를 오른쪽 소스코드로 변경해보자.

Refactor

Test

```
class GameTest {  
    @Test  
    void createGame() {  
        Game game = new Game();  
        assertNotNull(game);  
    }  
  
    @Test  
    void throwExceptionWhenInputIsNull() {  
        Game game = new Game();  
        try {  
            game.guess(null);  
        } catch (IllegalArgumentException e) {  
            // ...  
        }  
    }  
}
```

Refactor This

- 1 Rename... Ctrl+R, R
- Extract/Introduce
- 2 Introduce Constant... Ctrl+Alt+C
- 3 Introduce Field... Ctrl+R, F
- 4 Introduce Parameter... Ctrl+R, P
- 5 Introduce Functional Parameter... Ctrl+Alt+Shift+P
- 6 Inline Variable Ctrl+R, I

game 객체를
Field로 만들기

```
private Game game;
```

Generate

- Test Method
- Setup Method
- TearDown Method

Field 초기화를 위해
SetUp 메서드 추가

```
class GameTest {  
    private Game game;  
  
    @BeforeEach  
    void setUp() {  
        game = new Game();  
    }  
  
    @Test  
    void createGame() {  
        assertNotNull(game);  
    }  
  
    @Test  
    void throwExceptionWhenInputIsNull() {  
        try {  
            game.guess(null);  
        } catch (IllegalArgumentException e) {  
            // ...  
        }  
    }  
}
```

Baseball Game 실습(02/06)

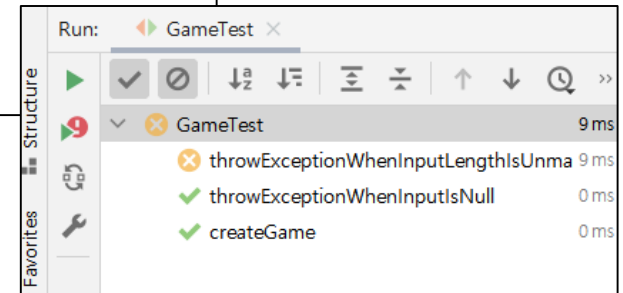
Red – 입력값 자리수가 세 자리가 아닐 경우 테스트케이스 작성

RED

```
@Test
void throwExceptionWhenInputLengthIsUnmatched() {
    assertThrows(IllegalArgumentException.class, () -> {
        game.guess("12");
    });
}
```

Test

새로운 테스트 케이스를 추가한다.
세 자리 숫자가 아니므로 Exception 발생해야한다.



세 자릿수 검사 코드 추가.

Green – Game 로직 추가

GREEN

```
public class Game {  
    public void guess(String guessNumber) {  
        if (guessNumber == null) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```

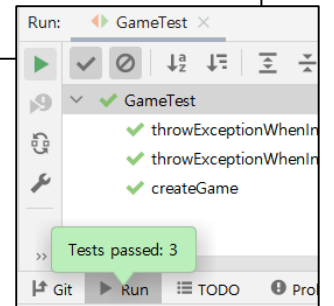
Production

세 자리인지 검사하는 코드 추가하기



```
public class Game {  
    public void guess(String guessNumber) {  
        if (guessNumber == null) {  
            throw new IllegalArgumentException();  
        }  
        if (guessNumber.length() != 3) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```

Green을
눈으로 확인한다.



expected를 try ~ catch로 변경

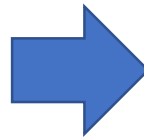
중복코드 제거를 위한 절차 1

Refactor

Test

```
@Test
void throwExceptionWhenInputIsNull() {
    assertThrows(IllegalArgumentException.class, () -> {
        game.guess(null);
    });
}
```

```
@Test
void throwExceptionWhenInputLengthIsUnmatched() {
    assertThrows(IllegalArgumentException.class, () -> {
        game.guess("12");
    });
}
```



```
@Test
void throwExceptionWhenInputIsNull() {
    assertThrows(IllegalArgumentException.class, () -> {
        game.guess(null);
    });
}
```

```
@Test
void throwExceptionWhenInputLengthIsUnmatched() {
    try {
        game.guess("12");
        fail();
    }
    catch (IllegalArgumentException e) {
    }
}
```

두 Test Case의 중복적인 부분을 메서드로
추출할 예정, 먼저 expected 대신 try catch로 변경

Extract variable (Introduce variable)

중복코드 제거를 위한 절차 2

Refactor

Test

```
@Test
public void throwExceptionWhenInputLengthIsUnmatched() {
    try {
        game.guess( guessNumber: "12");
        fail();
    }
    catch(IllegalArgumentException e) {
    }
}
```

Refactor This

- 1. Change Signature...
- 2. Move Instance Method...
- 3. Copy Class...
- 4. Introduce Variable...

"12" 문자열 값을 변수 도입하기
(Ctrl + Alt + Shift + T)
(Extract / Introduce Variable)

```
@Test
public void throwExceptionWhenInputLengthIsUnmatched() {
    try {
        String guessNumber = "12";
        game.guess(guessNumber);
        fail();
    }
    catch(IllegalArgumentException e) {
    }
}
```

한줄 위로 이동시키기
(Ctrl + Shift + UP)

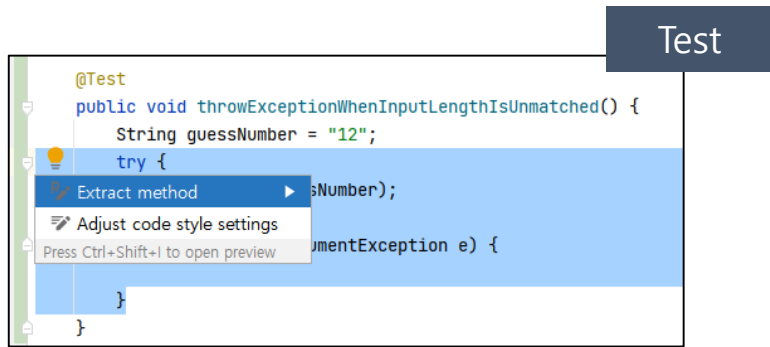
```
@Test
public void throwExceptionWhenInputLengthIsUnmatched() {
    String guessNumber = "12";
    try {
        game.guess(guessNumber);
        fail();
    } catch (IllegalArgumentException e) {
    }
}
```

포맷 맞추기
(Ctrl + Alt + L)

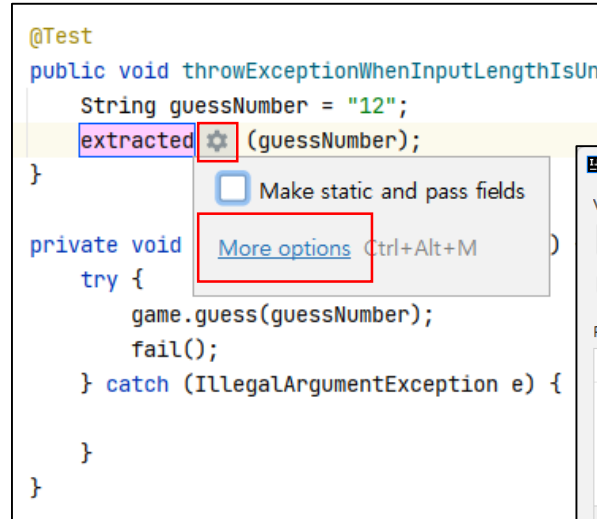
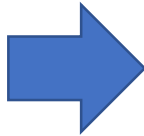
메서드 추출하기

중복코드 제거를 위한 절차 3

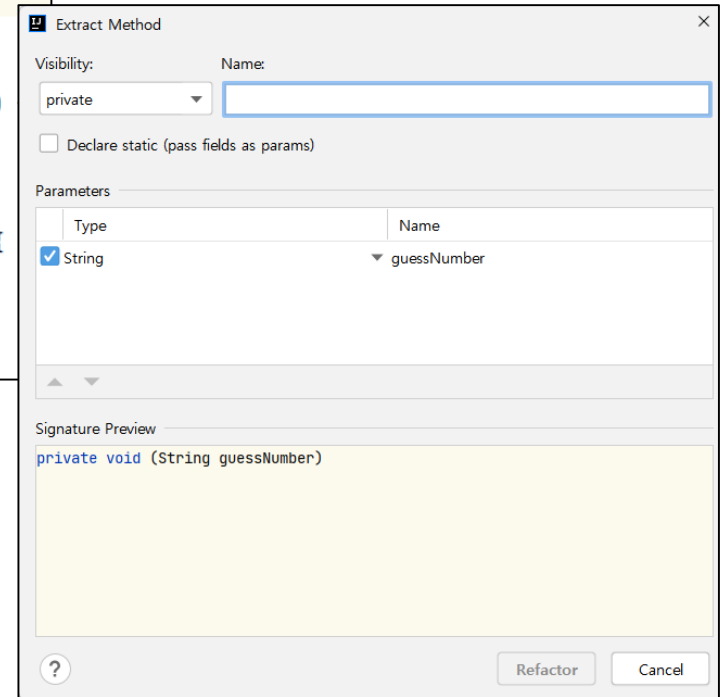
Refactor



try ~ catch 블록 잡고
Alt + Enter 누르고
Extract method



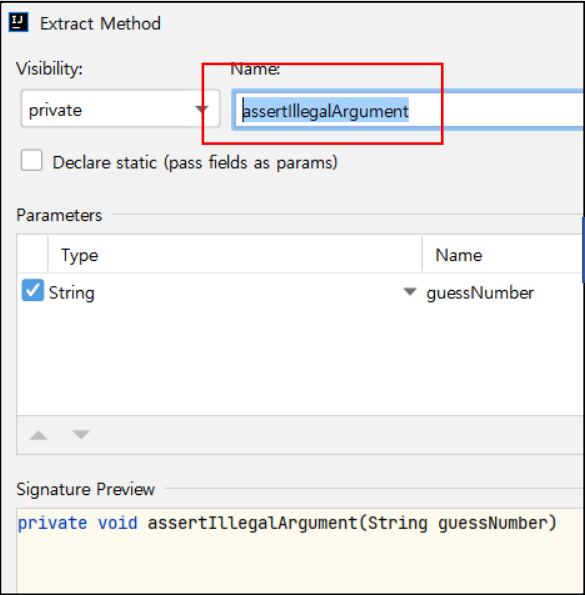
Extract Method 창 띄우기



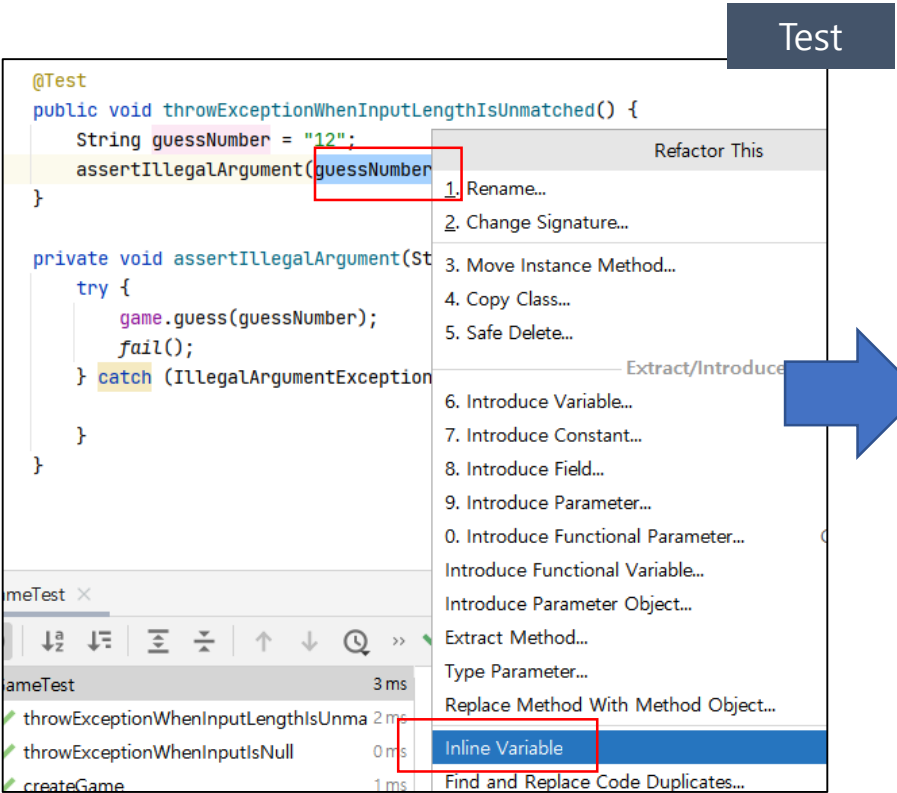
Extract Method 이후, Inline Variable 수행

중복코드 제거를 위한 절차 4

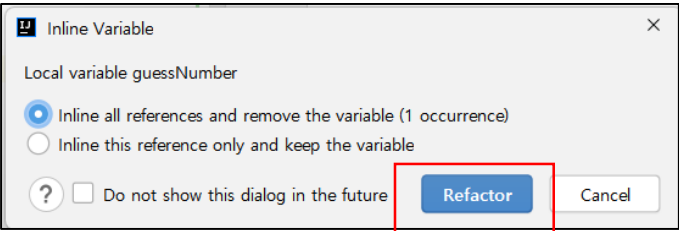
Refactor



assertIllegalArgument라는 메서드 명으로 메서드 추출



변수 명에 커서 놓고, Inline Variable 하기



Refactor 수행

코드 정리하기 (중복 코드 제거)

중복코드 제거를 위한 절차 5

Refactor

Test

```
@Test
void throwExceptionWhenInputIsNull() {
    assertThrows(IllegalArgumentException.class, () -> {
        game.guess(null);
    });
}
```

```
@Test
void throwExceptionWhenInputLengthIsUnmatched() {
    assertIllegalArgumentException("12");
}
```

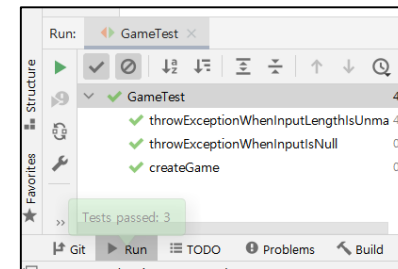
```
private void assertIllegalArgumentException(String guessNumber) {
    try {
        game.guess(guessNumber);
        fail();
    }
    catch (IllegalArgumentException e) {
    }
}
```

제작한 메서드로 변경

```
@Test
void throwExceptionWhenInputIsNull() {
    assertIllegalArgumentException(null);
}

@Test
void throwExceptionWhenInputLengthIsUnmatched() {
    assertIllegalArgumentException("12");
}

private void assertIllegalArgumentException(String guessNumber) {
    try {
        game.guess(guessNumber);
        fail();
    }
    catch (IllegalArgumentException e) {
    }
}
```



테스트 수행해보기

Unit Test 하나로 합치기

두 유닛테스트를 하나의 유닛테스트로 변경

Refactor

Test

```
@Test
public void throwExceptionWhenInputIsNull() {
    assertIllegalArgumentException( guessNumber: null);
}

@Test
public void throwExceptionWhenInputLengthIsUnmatched() {
    assertIllegalArgumentException( guessNumber: "12");
}
```

```
private void assertIllegalArgumentException(String guessNumber) {
    try {
        game.guess(guessNumber);
        fail();
    } catch (IllegalArgumentException e) {
    }
}
```

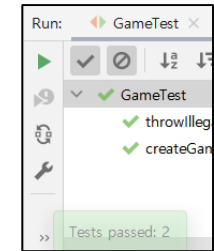
```
private void assertIllegalArgumentException(String guessNumber) {
    try {
        game.guess(guessNumber);
        fail();
    } catch (IllegalArgumentException e) {
    }
}
```

```
@Test
public void throwIllegalArgumentExceptionInvalidInput() {
    assertIllegalArgumentException( guessNumber: null);
    assertIllegalArgumentException( guessNumber: "12");
    assertIllegalArgumentException( guessNumber: "1234");
}
```

같은 역할을 하는 유닛테스트

메서드 명 변경
"1234" 테스트 케이스 추가.

테스트 수행해보기



Baseball Game 실습(03/06)

Red – 입력값에 숫자 외의 문자가 입력될 경우 테스트케이스 작성

RED

Test

```
@Test
public void throwIllegalArgumentExceptionInvalidInput() {
    assertIllegalArgumentException(guessNumber: null);
    assertIllegalArgumentException(guessNumber: "12");
    assertIllegalArgumentException(guessNumber: "1234");
}

private void assertIllegalArgumentException(String guessNumber) {
    try {
        game.guess(guessNumber);
        fail();
    } catch (IllegalArgumentException e) {
    }
}
```

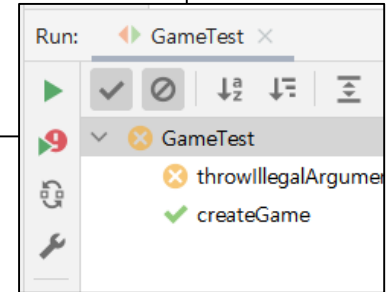
```
private void assertIllegalArgumentException(String guessNumber) {
    try {
        game.guess(guessNumber);
        fail();
    } catch (IllegalArgumentException e) {
    }
}
```

```
@Test
public void throwIllegalArgumentExceptionInvalidInput() {
    assertIllegalArgumentException(guessNumber: null);
    assertIllegalArgumentException(guessNumber: "12");
    assertIllegalArgumentException(guessNumber: "1234");
}
```

소스코드 블록 잡고,
Ctrl + Shift + Up

```
@Test
public void throwIllegalArgumentExceptionInvalidInput() {
    assertIllegalArgumentException(guessNumber: null);
    assertIllegalArgumentException(guessNumber: "12");
    assertIllegalArgumentException(guessNumber: "1234");
    assertIllegalArgumentException(guessNumber: "12s");
}
```

Invalid Input Case 추가하기 "12s"
그리고 Fail을 눈으로 확인한다.



'0' ~ '9' 문자열만 허용하도록 구현

Green – Game 로직 수정

GREEN

Production

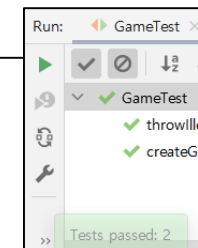
```
public class Game {  
    public void guess(String guessNumber) {  
        if (guessNumber == null) {  
            throw new IllegalArgumentException();  
        }  
  
        if (guessNumber.length() != 3) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```

기존 소스코드



```
public class Game {  
    public void guess(String guessNumber) {  
        if (guessNumber == null) {  
            throw new IllegalArgumentException();  
        }  
  
        if (guessNumber.length() != 3) {  
            throw new IllegalArgumentException();  
        }  
  
        for (char number : guessNumber.toCharArray()) {  
            if (number < '0' || number > '9') {  
                throw new IllegalArgumentException();  
            }  
        }  
    }  
}
```

'0' ~ '9' 외 문자열은
모두 Exception 처리한다.
Green을 눈으로 확인한다.



리팩토링 할 내용 없음

따로 Refactoring 할 부분이 없어,
이번 단계는 생략함 (Commit 안해도 됨)

Test

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class GameTest {

    private Game game;

    @BeforeEach
    void setUp() {
        game = new Game();
    }

    @Test
    void createGame() {
        assertNotNull(game);
    }

    private void assertIllegalArgumentException(String guessNumber) {
        try {
            game.guess(guessNumber);
            fail();
        }
        catch (IllegalArgumentException e) {
        }
    }

    @Test
    void throwIllegalArgumentExceptionInvalidInput() {
        assertIllegalArgumentException(null);
        assertIllegalArgumentException("12");
        assertIllegalArgumentException("1234");
        assertIllegalArgumentException("12s");
    }
}
```

Refactor

Production

```
public class Game {
    public void guess(String guessNumber) {
        if (guessNumber == null) {
            throw new IllegalArgumentException();
        }

        if (guessNumber.length() != 3) {
            throw new IllegalArgumentException();
        }

        for (char number : guessNumber.toCharArray()) {
            if (number < '0' || number > '9') {
                throw new IllegalArgumentException();
            }
        }
    }
}
```

Baseball Game 실습(04/06)

Red – 입력값에 중복된 숫자가 입력될 경우 테스트케이스 작성

RED

Test

```
@Test  
public void throwIllegalArgumentExceptionInvalidInput() {  
    assertIllegalArgumentException(guessNumber: null);  
    assertIllegalArgumentException(guessNumber: "12");  
    assertIllegalArgumentException(guessNumber: "1234");  
    assertIllegalArgumentException(guessNumber: "12s");  
    assertIllegalArgumentException(guessNumber: "121");  
}
```

같은 숫자가 들어있는 Invalid Case 추가
Fail을 눈으로 확인하자.

Run: ▶ GameTest ✕

Test Case	Result
throwIllegalArgumentExceptionInvalidInput	Fail
createGame	Pass

Tests failed: 1, passed: 1

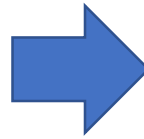
중복 숫자 값이 있는지 검사하는 코드 추가.

Green – Game 로직 수정

GREEN

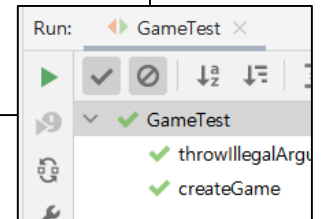
Production

```
public class Game {  
    public void guess(String guessNumber) {  
        if (guessNumber == null) {  
            throw new IllegalArgumentException();  
        }  
  
        if (guessNumber.length() != 3) {  
            throw new IllegalArgumentException();  
        }  
  
        for (char number : guessNumber.toCharArray()) {  
            if (number < '0' || number > '9') {  
                throw new IllegalArgumentException();  
            }  
        }  
    }  
}
```



```
public class Game {  
    public void guess(String guessNumber) {  
        if (guessNumber == null) {  
            throw new IllegalArgumentException();  
        }  
  
        if (guessNumber.length() != 3) {  
            throw new IllegalArgumentException();  
        }  
  
        for (char number : guessNumber.toCharArray()) {  
            if (number < '0' || number > '9') {  
                throw new IllegalArgumentException();  
            }  
        }  
  
        if (guessNumber.charAt(0) == guessNumber.charAt(1)  
            || guessNumber.charAt(0) == guessNumber.charAt(2)  
            || guessNumber.charAt(1) == guessNumber.charAt(2)) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```

모든 케이스를 검사하는 코드를 추가 후,
Green을 눈으로 확인하자.



메서드 추출하기 1

Refactoring – Game 클래스 extract method

Refactor

Production

```
if (guessNumber.charAt(0) == guessNumber.charAt(1)
    || guessNumber.charAt(0) == guessNumber.charAt(2)
    || guessNumber.charAt(1) == guessNumber.charAt(2)) {
    throw new IllegalArgumentException();
}
```

- Flip '=='
- Extract method
- Replace '||' with '&&'
- Add clarifying parentheses
- Remove braces from 'if' stat

if 내부 조건문만 블록을 잡는다.
Alt + Enter를 눌러 "Extract Method" 선택



```
if (isDuplicatedNumber(guessNumber)) {
    throw new IllegalArgumentException();
}

private boolean isDuplicatedNumber(String guessNumber) {
    return guessNumber.charAt(0) == guessNumber.charAt(1)
        || guessNumber.charAt(0) == guessNumber.charAt(2)
        || guessNumber.charAt(1) == guessNumber.charAt(2);
}
```

메서드명 : isDuplicatedNumber
그 후, Test로 잘 고쳐짐을 확인한다.

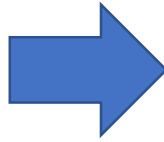
Run: GameTest
GameTest
✓ throwIllegal
✓ createGam
Tests passed: 2
Git Run TOD

메서드 추출하기 2

Refactoring – Game 클래스 extract method

Production

```
public class Game {
    public void guess(String guessNumber) {
        if (guessNumber == null) {
            throw new IllegalArgumentException();
        }
        if (guessNumber.length() != 3) {
            throw new IllegalArgumentException();
        }
        for (char number : guessNumber.toCharArray()) {
            if (number < '0' || number > '9') {
                throw new IllegalArgumentException();
            }
        }
        if (isDuplicatedNumber(guessNumber)) {
            throw new IllegalArgumentException();
        }
    }
}
```



```
public class Game {
    public void guess(String guessNumber) {
        assertIllegalArgument(guessNumber);
    }

    private void assertIllegalArgument(String guessNumber) {
        if (guessNumber == null) {
            throw new IllegalArgumentException();
        }

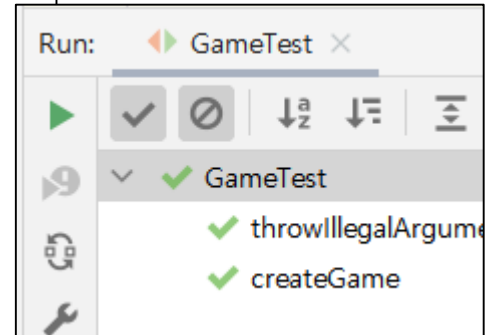
        if (guessNumber.length() != 3) {
            throw new IllegalArgumentException();
        }

        for (char number : guessNumber.toCharArray()) {
            if (number < '0' || number > '9') {
                throw new IllegalArgumentException();
            }
        }

        if (isDuplicatedNumber(guessNumber)) {
            throw new IllegalArgumentException();
        }
    }

    private boolean isDuplicatedNumber(String guessNumber) {
        return guessNumber.charAt(0) == guessNumber.charAt(1)
            || guessNumber.charAt(0) == guessNumber.charAt(2)
            || guessNumber.charAt(1) == guessNumber.charAt(2);
    }
}
```

메서드명 : assertIllegalArgument()
그 후, Test로 잘 고쳐짐을 확인한다.

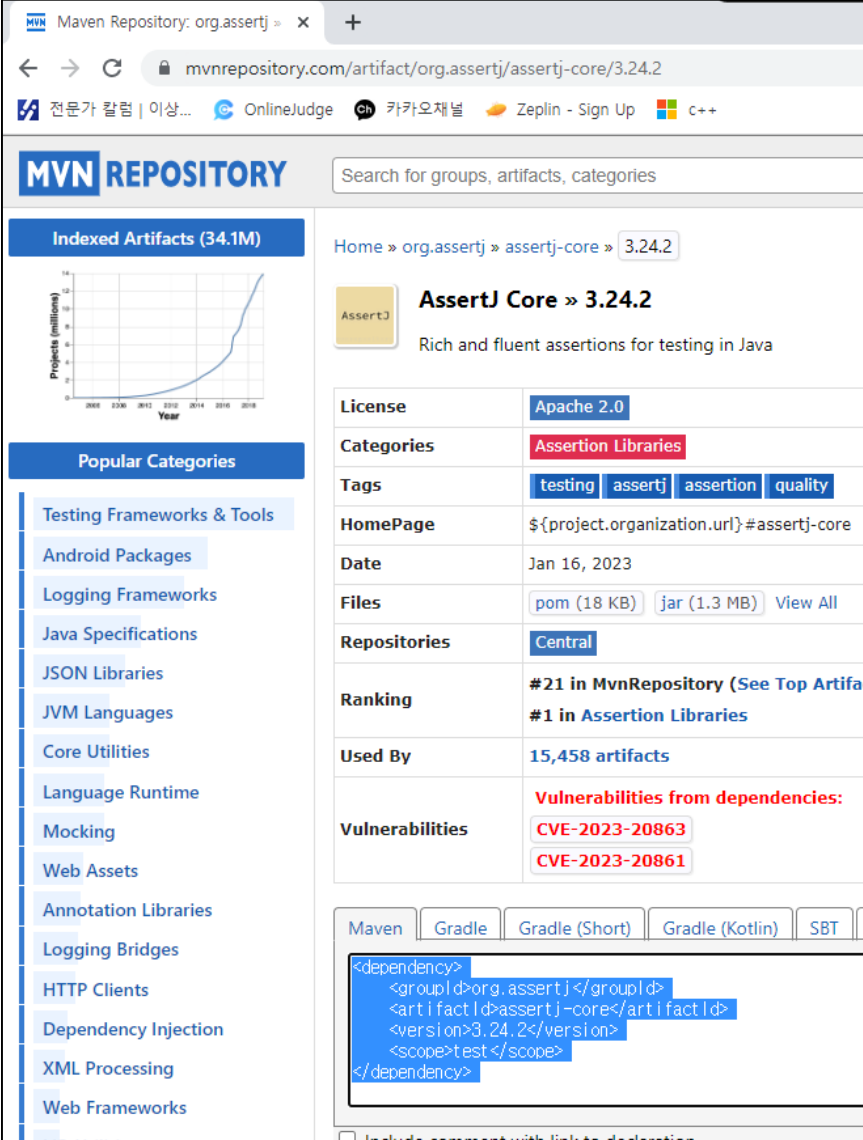


Refactor

MVN Repo 에서 AssertJ 설치

pom.xml 파일에 dependency 를 추가하여 AssertJ Core를 설치한다.

그 동안 테스트 코드 작성을 위해 AssertEquals을 써 왔지만, 이제부터 AssertJ Library의 AssertThat을 사용해본다.



Maven Repository: org.assertj » x +

mvnrepository.com/artifact/org.assertj/assertj-core/3.24.2

전문가 칼럼 | 이상... OnlineJudge 카카오채널 Zeplin - Sign Up c++

MVN REPOSITORY Search for groups, artifacts, categories

Indexed Artifacts (34.1M)

Popular Categories

- Testing Frameworks & Tools
- Android Packages
- Logging Frameworks
- Java Specifications
- JSON Libraries
- JVM Languages
- Core Utilities
- Language Runtime
- Mocking
- Web Assets
- Annotation Libraries
- Logging Bridges
- HTTP Clients
- Dependency Injection
- XML Processing
- Web Frameworks

Home » org.assertj » assertj-core » 3.24.2

AssertJ Core » 3.24.2
Rich and fluent assertions for testing in Java

License: Apache 2.0

Categories: Assertion Libraries

Tags: testing, assertj, assertion, quality

HomePage: \${project.organization.url}#assertj-core

Date: Jan 16, 2023

Files: pom (18 KB), jar (1.3 MB), View All

Repositories: Central

Ranking: #21 in MvnRepository (See Top Artifacts), #1 in Assertion Libraries

Used By: 15,458 artifacts

Vulnerabilities: CVE-2023-20863, CVE-2023-20861

Maven Gradle Gradle (Short) Gradle (Kotlin) SBT

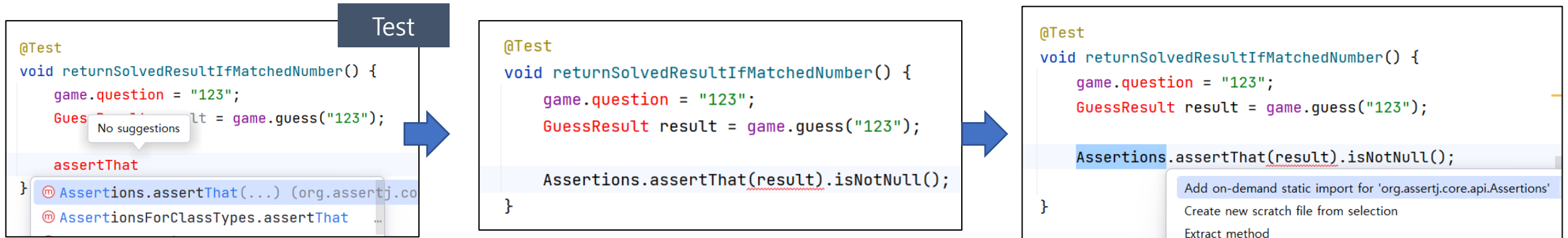
```
<dependency>
  <groupId>org.assertj</groupId>
  <artifactId>assertj-core</artifactId>
  <version>3.24.2</version>
  <scope>test</scope>
</dependency>
```

☐ Include comment with link to declaration

Baseball Game 실습(05/06)

Red – 숫자 세 개가 전부 일치할 경우 테스트 케이스 작성

RED



새로운 테스트 케이스를 작성한다.
assertThat이라고 쓰고 Ctrl + Space 2회 누르기
그리고 첫 번째 메뉴 선택하기
(자동으로 import문이 추가된다.)

Assertions 을 매번 입력 하지 않도록
"Assertions" 을 블록 잡은 후,
Alt + Enter 누르고 static import을 선택한다.
(자동으로 static import 문이 추가된다.)

Fail 테스트 코드 작성

AssertJ 를 이용한
테스트 코드 작성하기

```
1 import org.assertj.core.api.Assertions;
2 import org.junit.jupiter.api.BeforeEach;
3 import org.junit.jupiter.api.Test;
4
5 import static org.assertj.core.api.Assertions.*;
6 import static org.junit.jupiter.api.Assertions.*;
7
```

자동으로 추가된 import 문과 static import 문

```
@Test
void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("123");

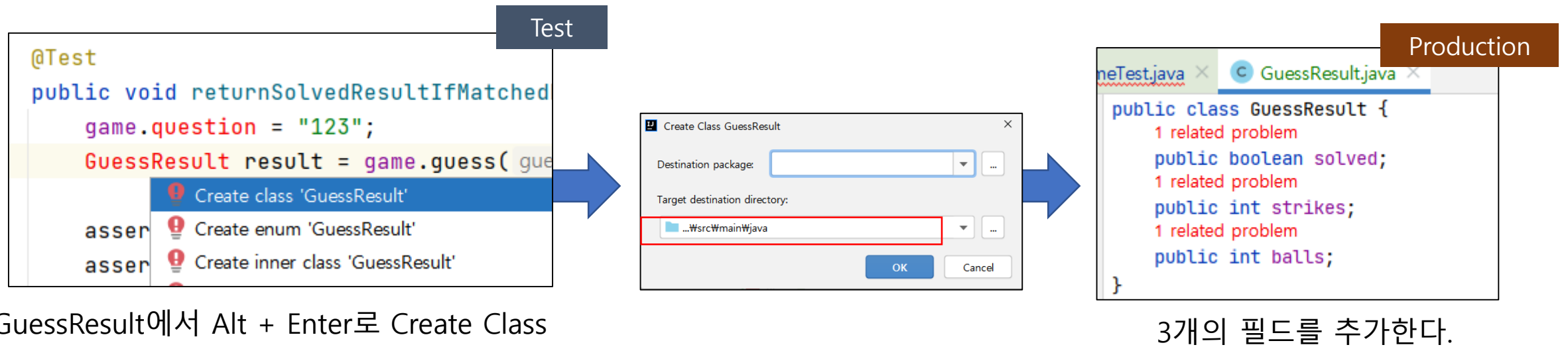
    //JUnit 5 + AssertJ
    assertThat(result).isNotNull();
    assertThat(result.solved).isEqualTo(true);
    assertThat(result.strikes).isEqualTo(3);
    assertThat(result.balls).isEqualTo(0);
}
```

테스트코드를 모두 완성한다.

GuessResult Class 추가하기

필드 3개 public으로 생성

RED



Game Class 소스코드 변경

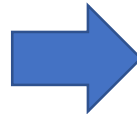
컴파일 에러가 나지 않도록 필드 추가 / 메서드 수정

RED

Production

```
public class Game {  
    1 related problem  
    public void guess(String guessNumber) {  
        assertIllegalArgument(guessNumber);  
    }  
  
    private void assertIllegalArgument(String guessNumber) {  
        if (guessNumber == null) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```

기존 소스코드



```
public class Game {  
    public String question;  
  
    public GuessResult guess(String guessNumber) {  
        assertIllegalArgument(guessNumber);  
        return null;  
    }  
  
    private void assertIllegalArgument(String guessNumber) {  
        if (guessNumber == null) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```

question : 정답 문자열이 담길 필드
GuessResult로 결과를 return 한다.

Fail 확인하기

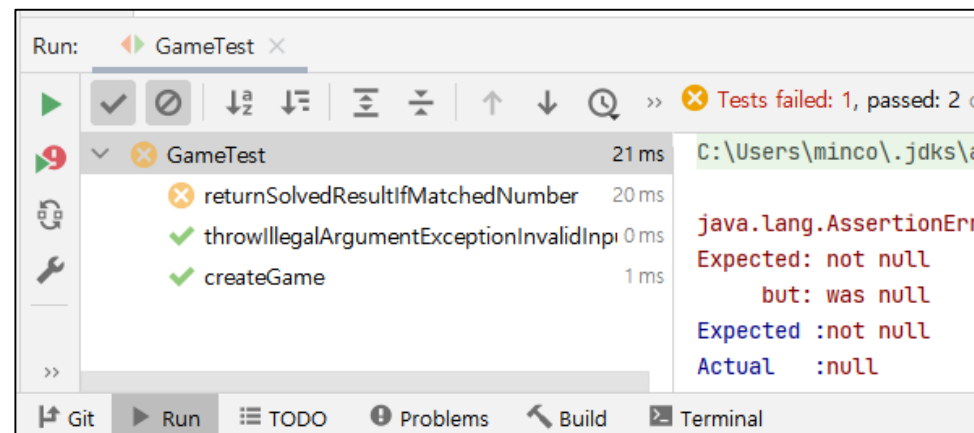
수정 완료

RED

```
@Test
void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("123");

    assertThat(result).isNotNull();
    assertThat(result.solved).isEqualTo(true);
    assertThat(result.strikes).isEqualTo(3);
    assertThat(result.balls).isEqualTo(0);
}
```

Test



완성된 테스트 케이스

Fail을 눈으로 확인한다.
null이 나왔으므로 Fail 이 발생했다.

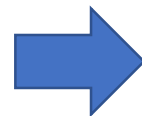
Game.java 파일 수정

Green – Game 로직 수정

GREEN

```
public class Game {  
  
    public String question;  
  
    public GuessResult guess(String guessNumber) {  
        assertIllegalArgument(guessNumber);  
        return null;  
    }  
}
```

Production



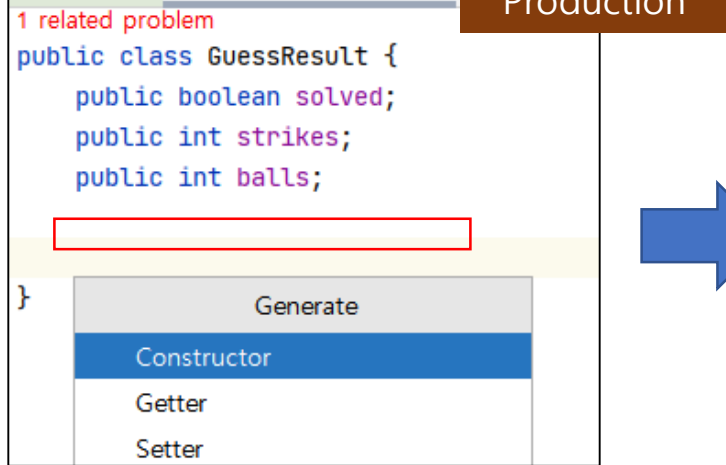
```
public class Game {  
  
    public String question;  
  
    public GuessResult guess(String guessNumber) {  
        assertIllegalArgument(guessNumber);  
        if (guessNumber.equals(question)) {  
            return new GuessResult(true, 3, 0);  
        }  
        else {  
            return null;  
        }  
    }  
}
```

정답인 경우, (true, 3, 0)
으로 리턴하기

기존 소스코드 (Game.java)

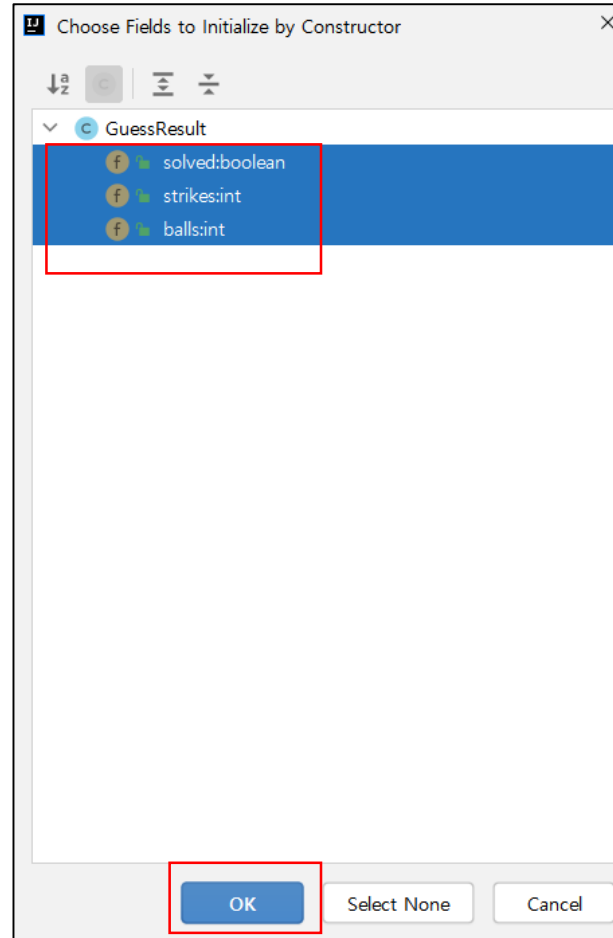
생성자 추가하기

Green – GuessResult 로직 수정

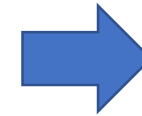


기존 소스코드 (GuessResult.java)

공백 입력 후 Alt + Insert,
Generate > Constructor 선택



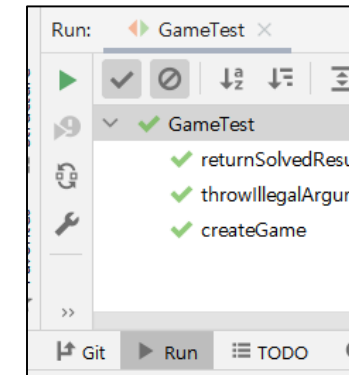
3개 필드 모두 선택하기



```
public class GuessResult {
    public boolean solved;
    public int strikes;
    public int balls;

    public GuessResult(boolean solved, int strikes, int balls) {
        this.solved = solved;
        this.strikes = strikes;
        this.balls = balls;
    }
}
```

생성자 추가 완료



Green을
눈으로 확인한다.

GuessResult.java 내용 Encapsulation

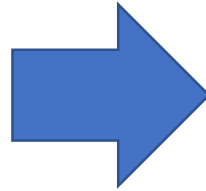
캡슐화를 위해

해당 필드 private로 변경하기

Refactor

```
public class GuessResult {  
    public boolean solved;  
    public int strikes;  
    public int balls;  
  
    public GuessResult(boolean solved, int strikes, int balls) {  
        this.solved = solved;  
        this.strikes = strikes;  
        this.balls = balls;  
    }  
}
```

기존 소스코드



```
public class GuessResult {  
    1 related problem  
    private boolean solved;  
    1 related problem  
    private int strikes;  
    1 related problem  
    private int balls;  
  
    public GuessResult(boolean solved,  
        this.solved = solved;  
        this.strikes = strikes;  
        this.balls = balls;  
    }  
}
```

public → private 로 변경

Getter 추가

Refactoring – Getter 추가하기

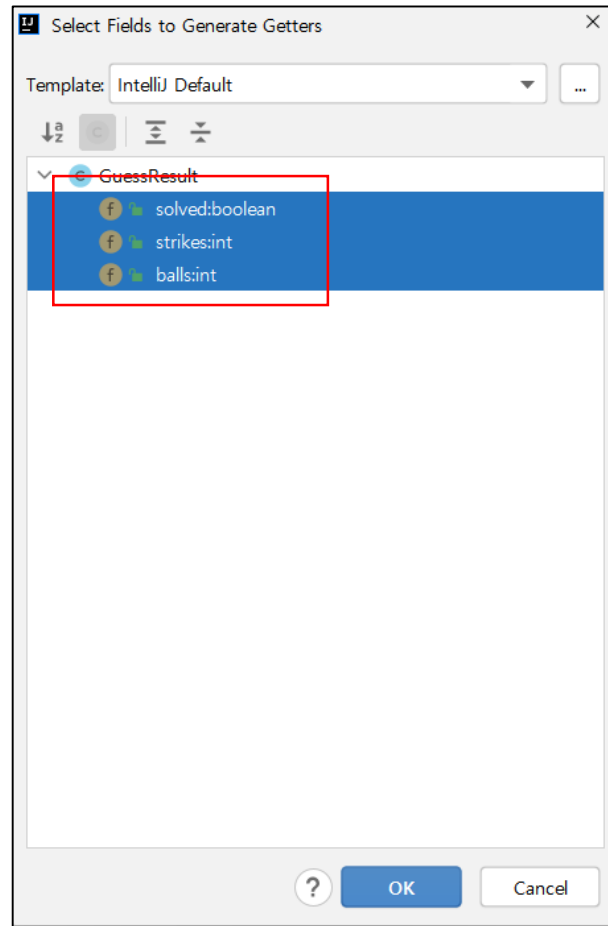
Production

```
1 public class GuessResult {
2     1 related problem
3     private boolean solved;
4     1 related problem
5     private int strikes;
6     1 related problem
7     private int balls;
8
9     public GuessResult(boolean solved,
10        this.solved = solved;
11        this.strikes = strikes;
12        this.balls = balls;
13 }
14
```

Generate
Constructor
Getter
Setter

기존 소스코드 (GuessResult.java)

공백 입력 후 Alt + Insert,
Generate > Getter 선택



3개 필드 모두 선택하기

Refactor

```
public class GuessResult {
    1 related problem
    private boolean solved;
    1 related problem
    private int strikes;
    1 related problem
    private int balls;

    public GuessResult(boolean solved,
        this.solved = solved;
        this.strikes = strikes;
        this.balls = balls;
    }

    public boolean isSolved() {
        return solved;
    }

    public int getStrikes() {
        return strikes;
    }

    public int getBalls() {
        return balls;
    }
}
```

Getter 추가 완료

리팩토링 - 캡슐화 하기

Getter로 접근하게끔 테스트 코드 수정하기

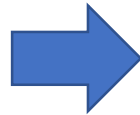
Refactor

Test

```
@Test
void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("123");

    assertThat(result).isNotNull();
    assertThat(result.solved).isEqualTo(true);
    assertThat(result.strikes).isEqualTo(3);
    assertThat(result.balls).isEqualTo(0);
}
```

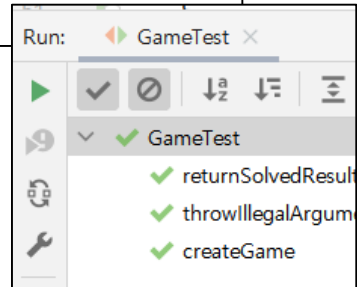
기존 소스코드



```
@Test
void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("123");

    assertThat(result).isNotNull();
    assertThat(result.isSolved()).isEqualTo(true);
    assertThat(result.getStrikes()).isEqualTo(3);
    assertThat(result.getBalls()).isEqualTo(0);
}
```

Getter 호출로 변경 후
Unit Test를 수행한다.



Baseball Game 실습(06/06)

Red – 숫자 세 개가 전부 일치하지 않을 경우 테스트 케이스 작성

Test

```
@Test
void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("123");

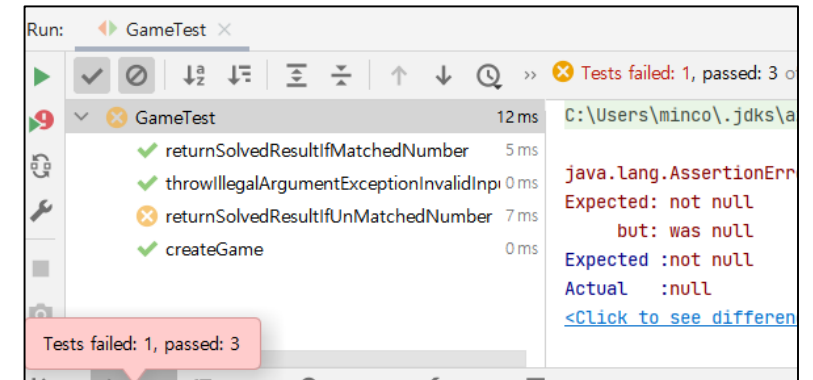
    assertThat(result).isNotNull();
    assertThat(result.isSolved()).isEqualTo(true);
    assertThat(result.getStrikes()).isEqualTo(3);
    assertThat(result.getBalls()).isEqualTo(0);
}
```

```
@Test
void returnSolvedResultIfUnMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("456");

    assertThat(result).isNotNull();
    assertThat(result.isSolved()).isEqualTo(false);
    assertThat(result.getStrikes()).isEqualTo(0);
    assertThat(result.getBalls()).isEqualTo(0);
}
```

테스트 케이스 추가하기

RED



Fail을 눈으로 확인한다.

전부 일치하지 않는 경우, 코드 구현

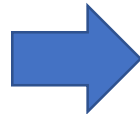
Green – Game 로직 수정

GREEN

Production

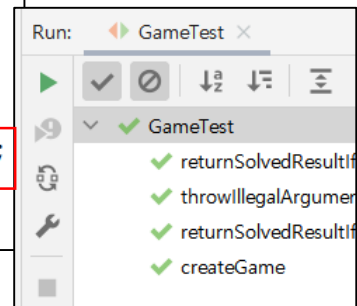
```
public class Game {  
  
    public String question;  
  
    public GuessResult guess(String guessNumber) {  
        assertIllegalArgument(guessNumber);  
        if (guessNumber.equals(question)) {  
            return new GuessResult( solved: true, strikes: 3, balls: 0);  
        }  
        else {  
            return null;  
        }  
    }  
}
```

기존 소스코드 (Game.java)



```
public class Game {  
  
    public String question;  
  
    public GuessResult guess(String guessNumber) {  
        assertIllegalArgument(guessNumber);  
        if (guessNumber.equals(question)) {  
            return new GuessResult( solved: true, strikes: 3, balls: 0);  
        }  
        else {  
            return new GuessResult( solved: false, strikes: 0, balls: 0);  
        }  
    }  
}
```

Fail시 Result 값 추가하기
그 후, Green을 눈으로 확인한다.



Extract Variable

Refactoring – Introduce Variable

Refactor

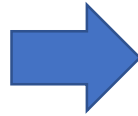
Test

```
@Test
void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("123");

    assertThat(result).isNotNull();
    assertThat(result.isSolved()).isEqualTo(true);
    assertThat(result.getStrikes()).isEqualTo(3);
    assertThat(result.getBalls()).isEqualTo(0);
}

@Test
void returnSolvedResultIfUnMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("456");

    assertThat(result).isNotNull();
    assertThat(result.isSolved()).isEqualTo(false);
    assertThat(result.getStrikes()).isEqualTo(0);
    assertThat(result.getBalls()).isEqualTo(0);
}
```



```
void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("123");

    boolean solved = true;
    int strikes = 3;
    int balls = 0;

    assertThat(result).isNotNull();
    assertThat(result.isSolved()).isEqualTo(solved);
    assertThat(result.getStrikes()).isEqualTo(strikes);
    assertThat(result.getBalls()).isEqualTo(balls);
}

@Test
void returnSolvedResultIfUnMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("456");

    boolean solved = false;
    int strikes = 0;
    int balls = 0;

    assertThat(result).isNotNull();
    assertThat(result.isSolved()).isEqualTo(solved);
    assertThat(result.getStrikes()).isEqualTo(strikes);
    assertThat(result.getBalls()).isEqualTo(balls);
}
```



하드코딩된 데이터에 변수 도입시키기

Extract method

Refactoring – extract method

Test

```
@Test
void returnSolvedResultIfUnMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess("456");

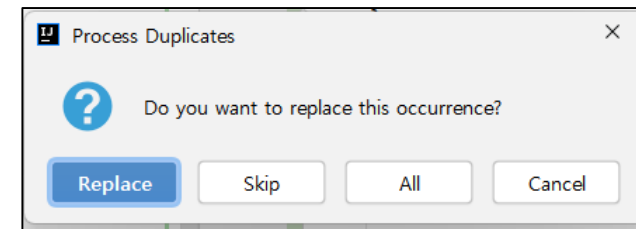
    boolean solved = false;
    int strikes = 0;
    int balls = 0;

    assertMatchedNumber(result, solved, strikes, balls);
}

private static void assertMatchedNumber(GuessResult result, boolean solved,
    int strikes, int balls) {
    assertThat(result).isNotNull();
    assertThat(result.isSolved()).isEqualTo(solved);
    assertThat(result.getStrikes()).isEqualTo(strikes);
    assertThat(result.getBalls()).isEqualTo(balls);
}
```

메서드 추출 (Ctrl + Alt + Shift + T > Extract Method)
메서드명 : assertMatchedNumber()

Refactor



중복코드 모두 Replace 하기

Inline Variable

변수들 모두 Inline 시키기

Refactor

Test

```
@Test
public void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess(guessNumber: "123");

    boolean solved = true;
    int strikes = 3;
    int balls = 0;

    assertMachedNumber(result, solved, strikes, balls);
}

@Test
public void returnSolvedResultIfUnMatchedNumber() {
    game.question = "123";
    GuessResult result = game.guess(guessNumber: "456");

    boolean solved = false;
    int strikes = 0;
    int balls = 0;

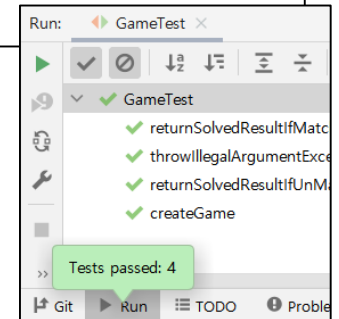
    assertMachedNumber(result, solved, strikes, balls);
}
```



```
@Test
public void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    assertMachedNumber(game.guess(guessNumber: "123"), solved: true, strikes: 3, balls: 0);
}

@Test
public void returnSolvedResultIfUnMatchedNumber() {
    game.question = "123";
    assertMachedNumber(game.guess(guessNumber: "456"), solved: false, strikes: 0, balls: 0);
}
```

Inline Variable
(변수 클릭 후, Ctrl + Alt + Shift + T를
사용하거나, 수동 변환)



메서드 추출 완료된 소스코드

Extract Method

추상화 레벨을 맞추기 위한 메서드 추출하기

Refactor

Test

```
@Test
public void returnSolvedResultIfMatchedNumber() {
    game.question = "123";
    assertMachedNumber(game.guess( guessNumber: "123"), solved: true, strikes: 3, balls: 0);
}

@Test
public void returnSolvedResultIfUnMatchedNumber() {
    game.question = "123";
    assertMachedNumber(game.guess( guessNumber: "456"), solved: false, strikes: 0, balls: 0);
}
```

메서드 추출하기
(수동으로 메서드 추출을 해주자.)

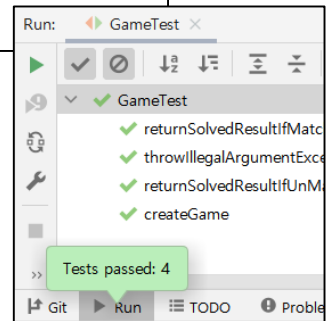


```
private void generateQuestion(String questionNumber) {
    game.question = questionNumber;
}

@Test
public void returnSolvedResultIfMatchedNumber() {
    generateQuestion( questionNumber: "123");
    assertMachedNumber(game.guess( guessNumber: "123"), solved: true, strikes: 3, balls: 0);
}

@Test
public void returnSolvedResultIfUnMatchedNumber() {
    generateQuestion( questionNumber: "123");
    assertMachedNumber(game.guess( guessNumber: "456"), solved: false, strikes: 0, balls: 0);
}
```

리팩토링 완성.
UnitTest 결과를 확인한다.



[도전] TDD 구현하기

1. 2 Strikes, 0 Ball 테스트 코드 추가 후,
Strikes를 구하는 코드 개발하기
2. 1 Stirkes, 2 Ball 테스트 코드 추가 후,
Balls을 구하는 코드 구현

감사합니다.